Tarefa 1 de Cálculo e Análise Numérica

O principio deste traballo é *"corrixir cara atrás, enviar cara adiante"*. A traballo estrutúrase en tres prazos que serán comunicados no Campus Virtual.

Prazo 1: proposta de traballos

Debes publicar durante o prazo 1 unha proposta de traballo no Foro que se habilitará no Campus Virtual.

Prazo 2: entrega interna

Trala finalización do prazo 1, no mesmo foro se publicará unha lista ordenada de estudantes, como a deste exemplo:

Nome
…
Persoa anterior
O teu nome
Persoa posterior
…

A lista considérase circular, é dicir, a persoa posterior á derradeira é a primeira da lista.

Durante o prazo 2 debes resolver o traballo proposto pola persoa posterior na lista e enviarlle a esta persoa a túa resolución de dito traballo. Polo mesmo motivo, deberás recibir da persoa anterior na lista a resolución da túa propia proposta de traballo.

Neste prazo 2, as comunicacións son internas entre estudantes, sen que os docentes reciban ningún documento. Se trala finalización deste prazo non recibes o envío previsto, si debes comunicarllo aos docentes.

Prazo 3: entrega externa

Durante o prazo 3 debes subir ao Campus Virtual dous ficheiros .ipynb, identificados do seguinte xeito, onde *ESTUDANTE* se refire ao teu nome no formato `Apelido1_Apelido2_Nome`:

1. *Traballo_ESTUDANTE_solucion_propia.ipynb*, coa resolución feita por ti da túa propia proposta.
2. *Traballo_ESTUDANTE_solucion_persoa_anterior.ipynb*, coa resolución feita pola anterior persoa na lista que recibiches no prazo 2. Debes subir o ficheiro recibido tal cual sen modificar ningún aspecto. Só podes (e debes) incluir unha cela de tipo Markdown ao final do ficheiro coa túa avaliación da solución recibida da persoa anterior.

Como se pode ver, neste envío non debes incluír a resolución feita por ti da proposta da persoa posterior, pois xa é ela a que lla remitirá aos docentes.

Avaliación

A puntuación máxima que podes obter é:

1. Resolución do teu propio traballo: 0,6 puntos, avaliado polos docentes.
2. Resolución do traballo posterior: 0,6 puntos, avaliado pola persoa posterior na lista. A avaliación proposta pola persoa posterior será revisada polos docentes para comprobar que a argumentación é correcta.
3. Corrección do traballo anterior: 0,3 puntos, avaliado polos docentes. Se algunha persoa avisou no prazo indicado de non ter recibido a resolución da persoa anterior, como este apartado non tería sentido, aumentarase o máximo do apartado 1. a 0,9 puntos.

Para que a corrección se considere válida, debes indicar os apartados nos que divides a corrección (por exemplo, presentación, dicotomía, Newton), a nota máxima de cada apartado e as razóns polas que lle asignaches a nota en cada un deles. A nota debe estar nunha escala de 0 a 0,6 puntos.

.

Instrucións detalladas para unha entrega

- Cada entrega debe ser un ficheiro .ipynb. Para incluir texto debes usar celas de tipo Markdown e, para incluir código en SageMath, celas de tipo Code. Non se admiten comentarios no código.
- Só se admiten ficheiros co Kernel de SageMath por defecto en Cocalc (9.7) ou a instalada nas aulas de informática (9.1).
- Se necesitas usar algunha función dada en clase, cópiaas nunha cela de tipo Code, sen alteralas, ao final do documento.
- Debes responder na orde indicada e anteceder cada resposta co número e título que se indican, nunha cela Markdown.
- Escribe as túas respostas coa letra de tamaño por defecto, como este documento e alí onde se indique, usa o máximo de liñas indicado. Non se admite xuntar varios comandos na mesma liña.
- Escribe o texto no editor; non pegues o texto como imaxe, pois non se poderá executar.
- As gráficas deben obterse das cela de código anterior; non se admiten capturas de pantalla.
- Cada resolución dun traballo que entregues (a outro estudante ou docente) debe conter a resposta a tódolos apartados indicados a continuación, e tan só a eses apartados.

Presentación

1. Nome da persoa que realiza a resolución do traballo (non o nome de quen propuxo o traballo).
2. Código para definir a función, incluidos os parámetros, e crear a gráfica no intervalo de traballo. O intervalo non poderá cambiar ao longo do traballo e só pode conter unha,

como moito dúas, raíces (máximo 10 liñas).

Dicotomía

3. Código para calcular da raíz por dicotomía que só devolva a aproximación da raíz e o número de iteracións (máximo 1 liña).
4. Código para crear a gráfica dos iterantes de dicotomía e os intervalos encaixados no intervalo de traballo (máximo 2 liñas).
5. Código para calcular o número máximo de iterantes (máximo 2 liñas).
6. Texto xustificando se o número de iteracións na práctica é coherente co número máximo de iterantes (máximo 1 liña).

Newton

7. Código para calcular da raíz por Newton que só devolva a aproximación da raíz e o número de iteracións (máximo 1 liña).
8. Código para crear a gráfica dos iterantes de Newton e as tanxente do método (máximo 2 liñas).
9. Código para crear a gráfica das constantes de converxencia, C_k (máximo 1 liña).
10. Texto xustificando se se dá a converxencia cuadrática neste caso (máximo 1 liña).

A continuación incluímos un exemplo de resolución.

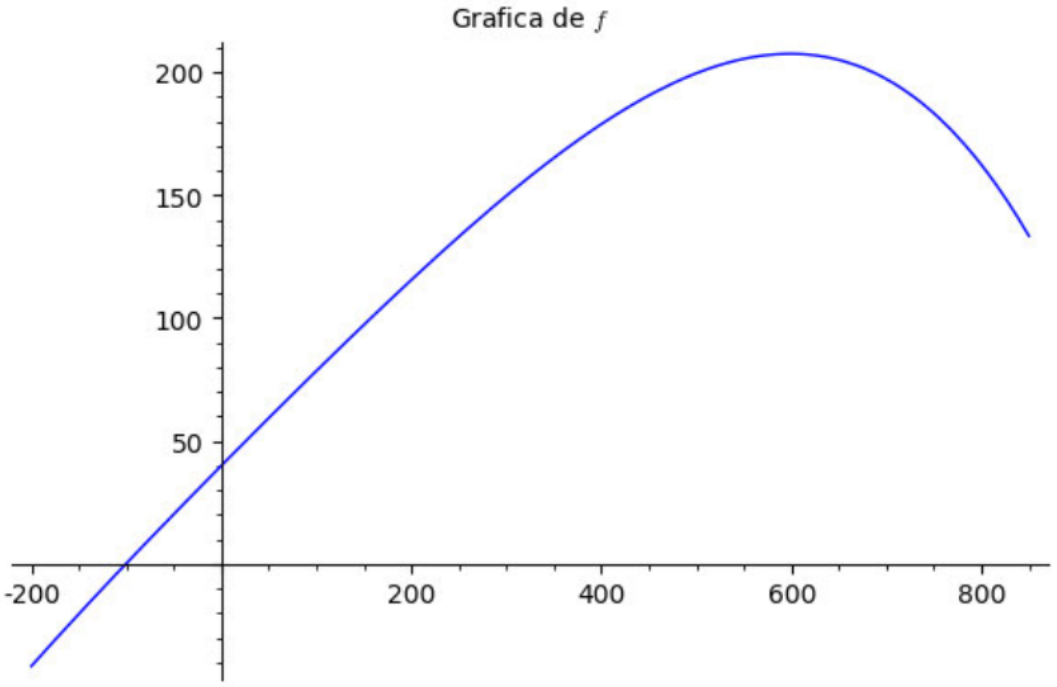
1. Persoa que realiza a resolución do traballo

Francisco José Pena Brage

2. Función, gráfica e intervalo de traballo

In [34]:
```
A = 3.90802e-3
B = 0.580195e-6
C = 4.27350e-12
f(x) = 100*(1+A*x-B*x^2-C*(x-100)*x^3)-60
a  = -200
b  =  850
plot(f, a, b, title='Grafica de $f$')
```

Out[34]:



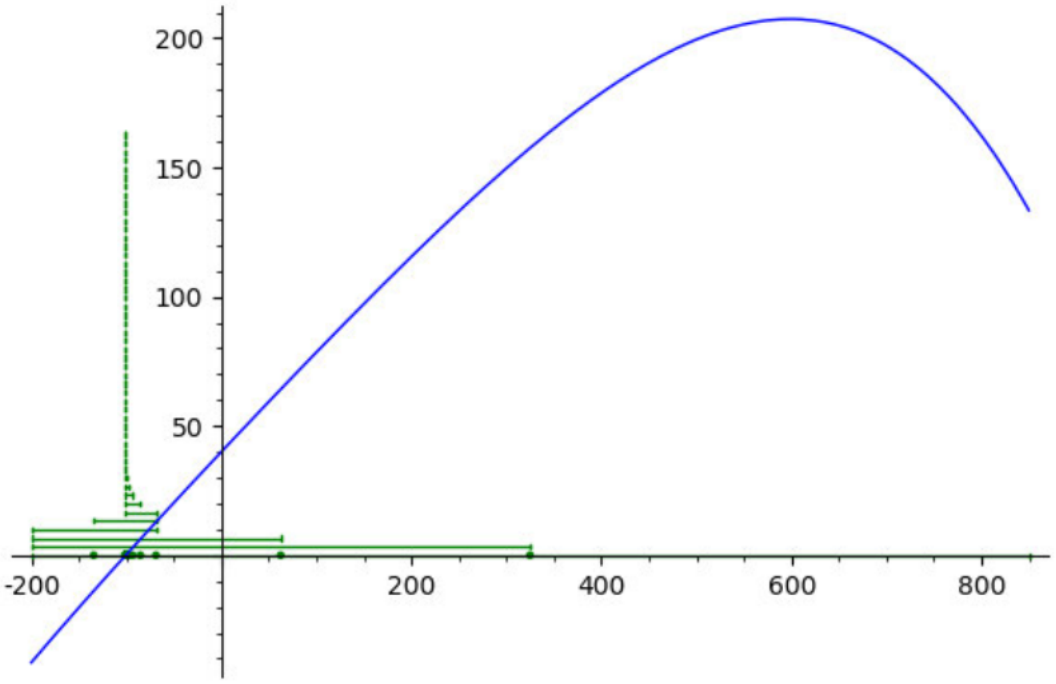
3. Cálculo da raíz por dicotomía

In [35]:
```
[redacted]
```

Out[35]: (-100.626785162330, 50)

4. Gráfica dos iterantes de dicotomía e os intervalos encaixados

In [36]:



5. Número máximo de iterantes

In [37]:

Out[37]: 50

6. Xustificación se o número de iteracións na práctica é coherente co número máximo de iterantes

Efectivamente, o número de iteracións (50) é menor ou igual ao número máximo (50).

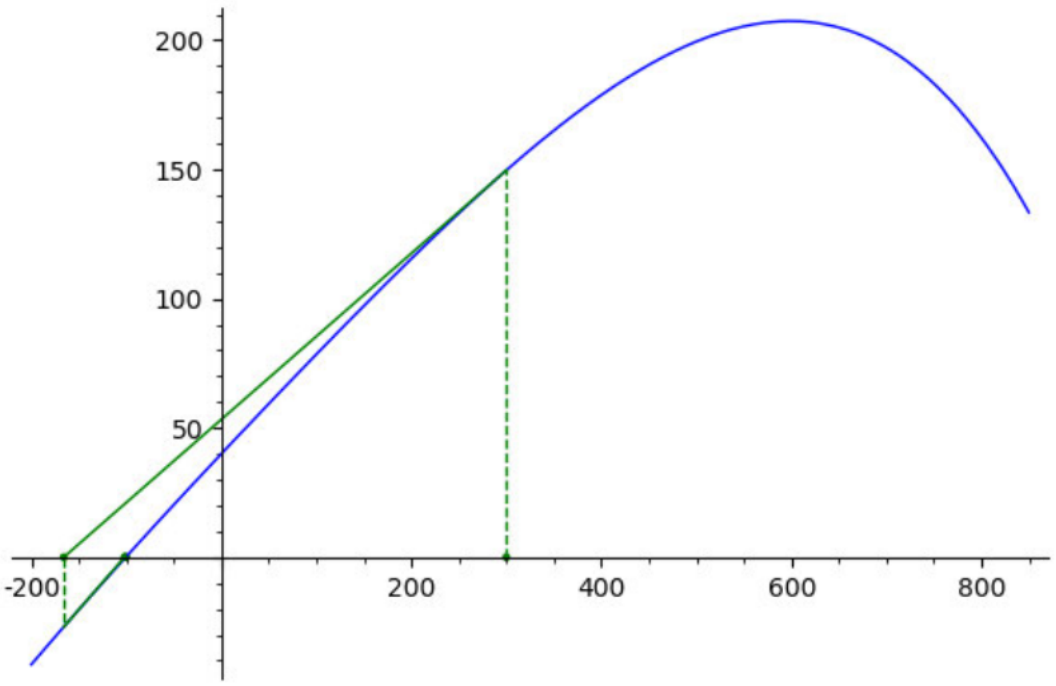
7. Cálculo da raíz por Newton

In [39]:

Out[39]: (-100.626785162331, 6)

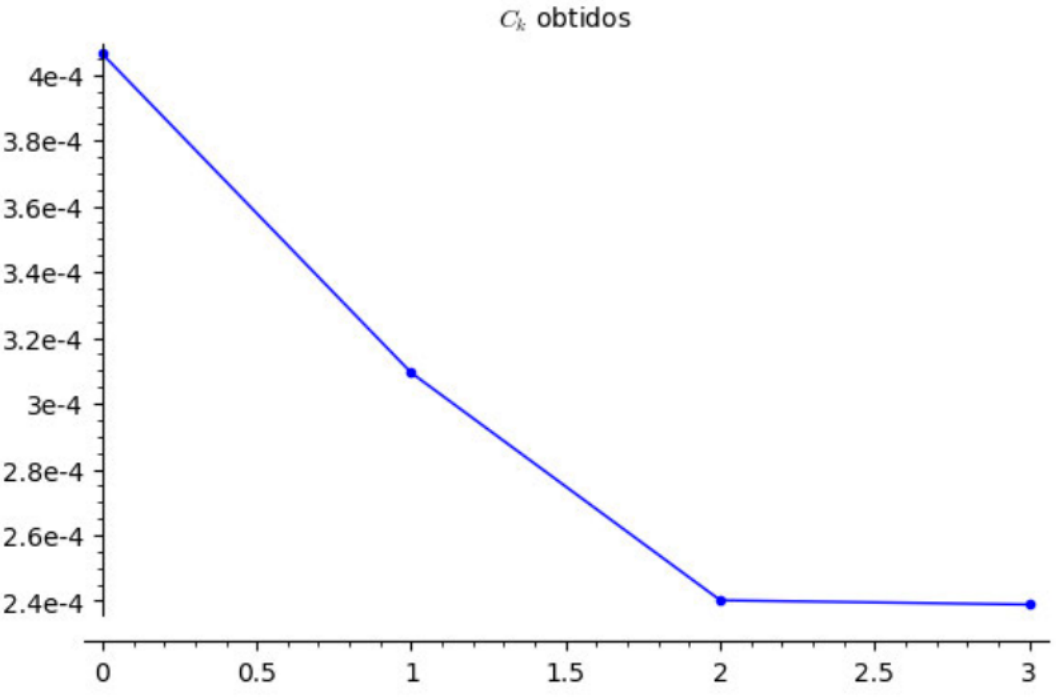
8. Gráfica dos iterantes de Newton e as tanxente do método

In [40]:



9. Gráfica das constantes de converxencia

In [41]:



10. Xustificación de se neste caso hai converxencia

Temos converxencia cuadrática porque os coeficientes converxen a $2.4e-4$ segundo medra o índice dos iterantes calculados.

ANEXO. Funcións de clase

In [42]:

```
def dicotomia(f, a, b, xtol=1e-12, rtol=1e-14):
    """DICOTOMIA    Calculo de raices por dicotomia."""
    r = a
    k = 0
    v = []
    I = []
    # comprobacions previas
    if abs(f(a)).n() < rtol:
        r = a.n(); return r,k,v,I
    elif abs(f(b)).n() < rtol:
        r = b.n(); return r,k,v,I
    elif (f(a)*f(b)).n() > 0:
        raise RuntimeError('Extremos de igual signo.')
    # calculo iterativo
    while abs(b - a) > xtol:
        r = (a + b).n() / 2
        k = k + 1
        v.append(r)
        I.append((a,b))
        if abs(f(r)).n() < rtol:
            return r,k,v,I
        elif (f(a)*f(r)).n() < 0:
            b = r
        else:
            a = r
    return r,k,v,I
```

In [43]:

```
def debuxa_dicotomia(f, a, b, v, I):
    """DEBUXA_DICOTOMIA    Visualiza intervalos de dicotomía."""
    p = plot(f, a, b) + point(zip(v,[0]*len(v)), color='green')
    h = (p.ymax() - p.ymin())/(1.5*len(v))
    show(sum(line([(a,h*i),    (b,h*i)],     color='green') + \
             line([(a,h*i-h/4),(a,h*i+h/4)], color='green') + \
             line([(b,h*i-h/4),(b,h*i+h/4)], color='green')   \
             for i, (a,b) in enumerate(I)) + p, xmin=a, xmax=b)
```

In [44]:

```
def newton(f, df, x0, xtol=1e-12, rtol=1e-14, M=100, errorM=True):
    """NEWTON    Calculo de raices por Newton."""
    v = [x0.n()]
    for k in range(M):
        dfx = df(v[k]).n()
        if abs(dfx) < rtol:
            raise RuntimeError('Derivada nula.')
        r = (v[k] - f(v[k]) / dfx).n()
        v.append(r)
        # test de parada
        if abs(r-v[k]) < xtol or abs(f(r)).n() < rtol:
            return r, len(v), v
    if errorM: raise RuntimeError('Excedido numero maximo de iteracions
    return r, len(v), v
```

In [45]:

```
def debuxa_newton(f, a, b, v):
    """DEBUXA_NEWTON    Visualiza tanxentes do metodo de Newton."""
    show(plot(f, a, b) + point(zip(v,[0]*len(v)), color='green') + \
    sum(line([(v[i],0),(v[i],f(v[i]))], color='green', linestyle='--')
    sum(line([(v[i-1],f(v[i-1])), (v[i],0)], color='green') for i in ra
```

In [46]:

```
def coef_newton(f, a, b, v):
    """COEF_NEWTON    Calcula os coeficientes de converxencia do metodo
    alpha = find_root(f, a, b, xtol=9.e-16, rtol=9.e-16) #aproximamos "
    pts = []
    for k in range(len(v)-1):
        if (alpha - v[k])^2 > RDF.epsilon():
            pts.append( (k, abs(alpha - v[k+1]) / (alpha - v[k])^2 ) )
    show(line(pts, marker = '.', title='$C_k$ obtidos'))
```

Avaliación

No caso de estar avaliando a resolución do teu traballo feita pola persoa anterior na lista, inclúe túa valoración nesta cela final de tipo Markdown.

In [ ]: